

Department of Computer Science

University of Engineering and Technology Lahore
Pakistan

هم نوا

Hamnawa

Accessible Document Reader for Visually Impaired Students

System Architecture Document

Subject: Software Engineering **Group No.:** 2

Submitted by:

Muhammad Sarim	2025-CS-191
Abdullah Akbar	2025-CS-212
Muhammad Junaid Amir	2025-CS-193
Muhammad Ahmad Chaudhry	2025-CS-184

Submitted to:

Dr. Maimoona Salam

October 31, 2026

Document Control

Project	Hamnawa (ہم نوا) — Accessible Document Reader for Visually Impaired Students
Document	System Architecture Document, version 0.1 (draft)
Course	Software Engineering, Department of Computer Science, UET Lahore
Group	Group 2
Supervisor	Dr. Maimoona Salam
Based on	Project Proposal v1.0; Functional and Non-Functional Requirements v0.1
Status	Draft for supervisor review

Revision History

Version	Date	Description	Author(s)
0.1	October 31, 2026	First draft of architecture, use cases, CRC cards and ERD.	Group 2
0.2	[date]	[After supervisor feedback]	[names]

At a glance. Hamnawa is a **client-side, layered web application** with a background document-processing pipeline. This document presents its architecture through the layered, processing, dynamic and deployment views, the use-case model, **16 CRC cards**, and an entity-relationship diagram of its data.

Contents

1 Introduction **4**

1.1 Purpose . . . 4
1.2 Scope . . . 4
1.3 Intended Readers . . . 4
1.4 Notation and Method . . . 4
1.5 References . . . 4

2 Architectural Drivers **5**

3 Architectural Overview **5**

3.1 Architectural Style . . . 5
3.2 Layered View . . . 6
3.3 Component Responsibilities . . . 6
3.4 Technology Choices (Proposed) . . . 8
3.5 Key Design Decisions . . . 9

4 Runtime and Deployment Views **10**

4.1 Document Processing Pipeline . . . 10
4.2 Dynamic View: Open a Document and Start Listening . . . 10
4.3 Deployment View . . . 11

5 Use-Case Model **11**

5.1 Actors . . . 11
5.2 System-Level Use-Case Diagram . . . 12
5.3 Use-Case Diagram with Include and Extend Relationships . . . 13
5.4 Use-Case Catalogue . . . 13

6 Class-Responsibility-Collaborator (CRC) Model **14**

6.1 How to Read the Cards . . . 14
6.2 Cards . . . 14
6.3 Collaboration Check . . . 19

7 Data Model: Entity-Relationship Diagram **20**

7.1 Data Overview . . . 20

7.2 ERD 20
7.3 Data Dictionary 21

8 Quality Attributes and Architectural Tactics 23

9 Traceability 23

9.1 Components to Requirements 23
9.2 Use Cases to Components 24

10 Architectural Risks and Open Issues 25

A Glossary 25

1 Introduction

1.1 Purpose

This document describes the architecture of Hamnawa: the major components, how they are organised and how they work together to satisfy the requirements. It records the key design decisions and the reasons behind them, and it contains the analysis and design models required by the course: the **use-case diagrams**, the **CRC (Class-Responsibility-Collaborator) cards** and the **entity-relationship diagram (ERD)**.

1.2 Scope

The architecture covers the complete product defined in the requirements document: document loading, extraction and structure recovery, spoken playback, navigation by keyboard and voice, bookmarks and resume, and settings. Out-of-scope items (OCR, image description, Urdu support, native mobile apps) are not designed here.

1.3 Intended Readers

The development team (to build the system), the supervisor (to review the design) and the testers (to derive integration tests from the interfaces and scenarios).

1.4 Notation and Method

- **Use-case diagrams** follow UML: actors outside the system boundary, use cases as ovals, *include* and *extend* as dashed arrows.
- **CRC cards** follow the class-based modelling technique described by Pressman and Maxim: each class lists its responsibilities and the collaborating classes.
- **ERD** uses entity boxes with attributes (PK = primary key, FK = foreign key) and cardinalities (1, 0..1, 0..*, 1..*).
- **Architecture views** are the layered view, the processing view, the dynamic (sequence) view and the deployment view.

1.5 References

[1] Hamnawa Project Proposal, version 1.0.

[2] Hamnawa Functional and Non-Functional Requirements, version 0.1.

[3] R. S. Pressman and B. R. Maxim, *Software Engineering: A Practitioner's Approach*, McGraw-Hill (architectural design; requirements modelling; class-based modelling and CRC cards).

[4] W3C, *WAI-ARIA Authoring Practices* and *WCAG 2.2*.

[5] Browser documentation for the Web Speech API (speech synthesis and speech recognition), Web Workers, and IndexedDB.

2 Architectural Drivers

The architecture is shaped by a small number of requirements that are hard to change later. They are the *drivers*; every major decision in Section 3.5 answers at least one of them.

Driver	Why it matters	Requirements
Accessibility first	The user cannot see the screen; every function must work by keyboard, speech and a screen reader. The interface must be semantic HTML, not a drawing surface.	NFR-A1 to A6, NFR-U1, U2, FR-27, FR-47
Privacy and zero cost	Course documents may be personal; there is no budget for servers. Processing on the user's device avoids both problems.	NFR-S1, S2, BR-05, cost constraint
Responsiveness	Parsing a large file must not freeze the keyboard or the speech. Commands must feel immediate.	NFR-P1 to P4
Robustness	Real course files are messy. A bad file must produce a spoken explanation, never a crash.	NFR-R1, NFR-S4, FR-03
Dependence on browser speech	Speech output and recognition come from the browser and differ between browsers. They may fail or be missing.	NFR-C1 to C3, FR-39, NFR-R3
Incremental delivery	The team delivers in four increments; each must run. Modules must be addable without rewriting earlier ones.	Process model, NFR-M1
Testability	Core logic must be testable without a browser or a microphone.	NFR-M1, M2

3 Architectural Overview

3.1 Architectural Style

Hamnawa combines three well-known styles:

1. **Layered architecture.** Four layers (presentation, application, domain and processing, infrastructure) where each layer uses only the layer below it. This keeps the interface separate from logic and makes the logic testable.
2. **Pipeline (pipes and filters).** A document flows through validate, extract, clean and

structure, and build model steps. Each step has one job and a defined input and output, so steps can be tested and replaced independently.

3. **Adapters at the edges.** Everything that depends on the browser (speech synthesis, speech recognition, storage) is hidden behind a small interface, so a missing or different browser feature affects one class only.

Components talk to each other by direct calls downward and by *events* upward (for example `documentReady`, `sentenceStarted`, `positionChanged`), so lower layers never depend on the interface.

3.2 Layered View

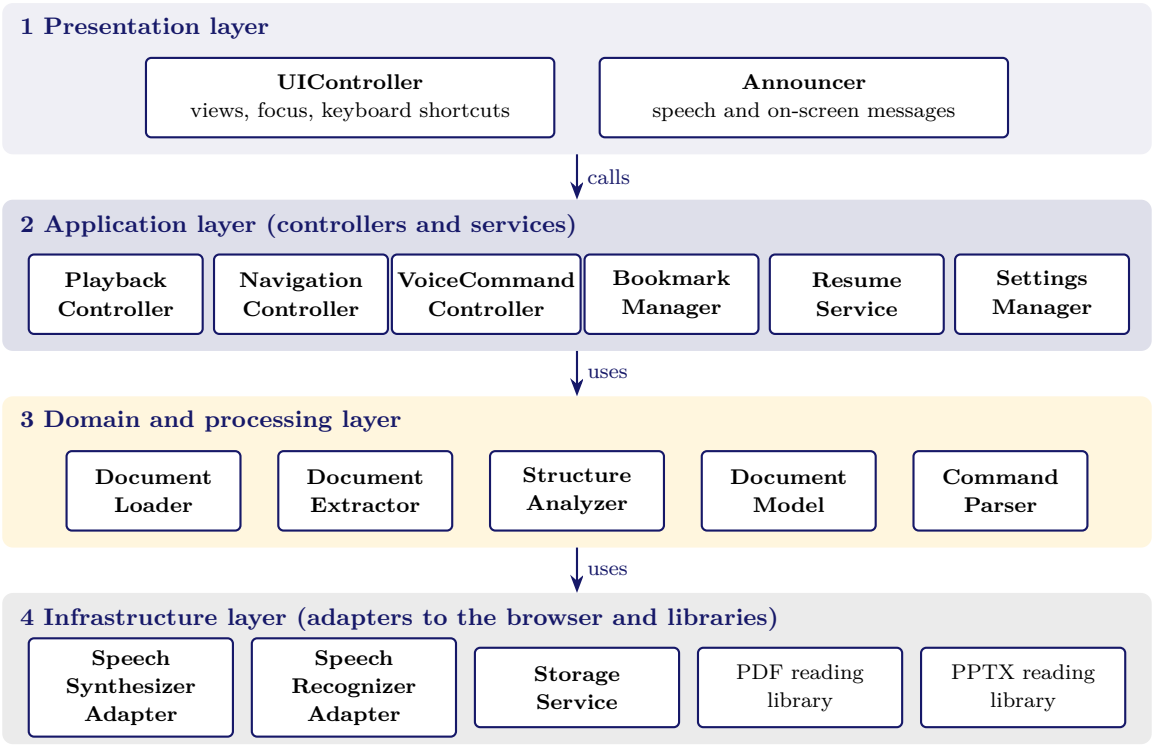


Figure 1: Layered view. Calls go downward; events travel upward. The browser's speech, storage and worker features are reached only through layer 4.

3.3 Component Responsibilities

Table below summarises each component, the layer it belongs to and the increment in which it is first delivered. The detailed responsibilities and collaborators are on the CRC cards in Section 6.

Component	Layer	Summary	Inc.
`UIController`	Presentation	Renders the three regions, manages focus and keyboard shortcuts, high contrast and text size, help.	1

Component	Layer	Summary	Inc.
`Announcer`	Presentation	Delivers every status and error message as speech and as on-screen text.	1
`PlaybackController`	Application	Controls reading: play, pause, repeat, skip, speed, voice, structural cues, continuous reading.	1
`NavigationController`	Application	Moves through the document by section, heading level, slide or page; where am I; search.	2
`VoiceCommandController`	Application	Switches the microphone on and off, listens, and turns recognised phrases into actions.	3
`BookmarkManager`	Application	Adds, lists, deletes and jumps to bookmarks of the open document.	3
`ResumeService`	Application	Saves the last reading position, offers to resume, and keeps the recent-documents list.	3
`SettingsManager`	Application	Loads, saves and applies user settings; erases all stored data on request.	3
`DocumentLoader`	Domain and proc.	Accepts a file, validates it, computes its fingerprint and runs the processing pipeline.	1
`DocumentExtractor`	Domain and proc.	Reads text, order and style information from PDF and PPTX files (one subclass for each).	1–2
`StructureAnalyzer`	Domain and proc.	Cleans the raw text and recovers headings, lists, paragraphs, slides and the outline.	2
`DocumentModel`	Domain and proc.	Holds the outline and content blocks and answers position and navigation queries.	1–2
`CommandParser`	Domain and proc.	Turns a recognised phrase into a command with parameters, or reports it as unknown.	3
`SpeechSynthesizerAdapter`	Infrastructure	Wraps the browser's text-to-speech and hides its differences and failures.	1

Component	Layer	Summary	Inc.
`SpeechRecognizerAdapter`	Infrastructure	Wraps the browser's speech recognition and the microphone permission.	3
`StorageService`	Infrastructure	Stores and retrieves records in the browser's local storage; reports quota and access errors.	3

3.4 Technology Choices (Proposed)

All choices are free and open source and are to be confirmed in the first increment (licence check: NFR-L1).

Concern	Choice	Reason
Language	JavaScript (or TypeScript)	Runs natively in the browser; one language for the whole product.
User interface	Semantic HTML, ARIA, CSS (no canvas)	Screen readers and keyboard focus work only with real elements.
PDF reading	pdf.js	Mature open-source library that exposes text with position and font data.
PPTX reading	Unzip and parse the XML inside the file	A PPTX file is a zip of XML; slide titles and text can be read directly.
Speech output and input	Web Speech API (speech synthesis, speech recognition)	Built into Chrome and Edge, free, no server.
Background processing	Web Worker	Keeps the interface and speech responsive while a file is parsed.
Local storage	IndexedDB or localStorage	Stores bookmarks, positions and settings only on the user's device.
Document fingerprint	Web Crypto (SHA-256)	Gives a stable identity for a file (BR-02).
Testing	A JavaScript test runner; NVDA; an accessibility checker	Unit tests for core logic; manual and automated accessibility checks.
Hosting and version control	Free static hosting; Git and GitHub	No server cost; history and review of changes.

3.5 Key Design Decisions

ID	Decision	Alternatives considered	Rationale and trade-off
D1	Process documents entirely in the browser.	Upload to a server and process there.	Protects privacy and costs nothing. Trade-off: limited by the user's device, so heavy work goes to a Web Worker. Revisit if Increment 1 shows poor speed.
D2	Layered style with a pipeline for documents.	Single large script; model-view-controller only.	Clear boundaries, easy unit tests, parallel work by the team. Trade-off: more files and interfaces.
D3	Wrap speech and storage in adapter classes.	Call browser APIs directly everywhere.	Isolates browser differences and failures (FR-39, NFR-R3) and lets tests use fake adapters.
D4	One format-independent document model.	Separate logic for PDF and PPTX.	Playback, navigation and bookmarks work the same for both formats; only extractors differ.
D5	Run extraction and structuring in a Web Worker.	Run on the main thread.	Keeps keys and speech responsive (NFR-P2). Trade-off: messages between threads must be designed.
D6	Identify documents by content fingerprint.	Use the file name.	Bookmarks survive renaming and copying (BR-02).
D7	Components communicate through events upward.	Direct calls both ways.	Lower layers stay independent of the interface; fewer circular dependencies.
D8	Sentence-by-sentence speech queue.	Hand the whole text to the speech engine.	Gives exact position for repeat, skip and resume, and avoids known browser problems with very long utterances.

4 Runtime and Deployment Views

4.1 Document Processing Pipeline

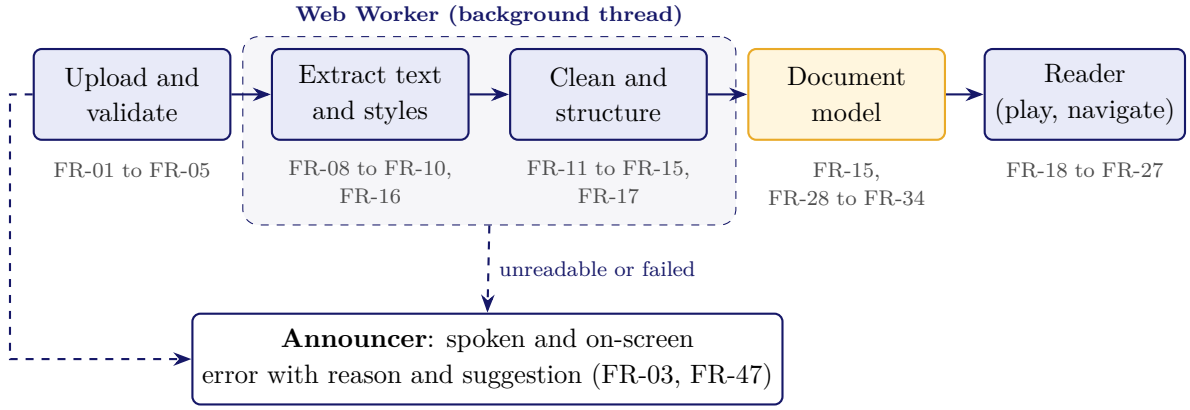


Figure 2: Processing view: a document flows left to right; any failure is routed to the Announcer so the user is always told what happened.

4.2 Dynamic View: Open a Document and Start Listening

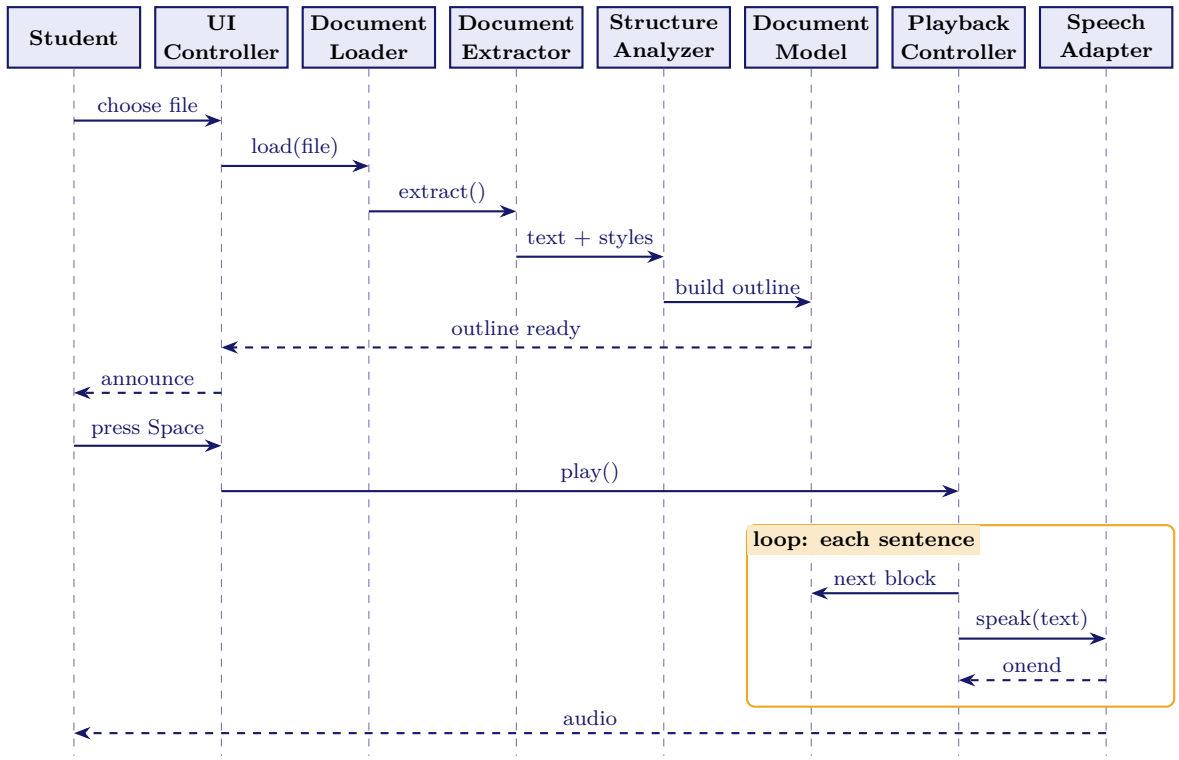


Figure 3: Sequence diagram for the main scenario (use cases UC-01 and UC-02). Dashed arrows are returns or events.

4.3 Deployment View

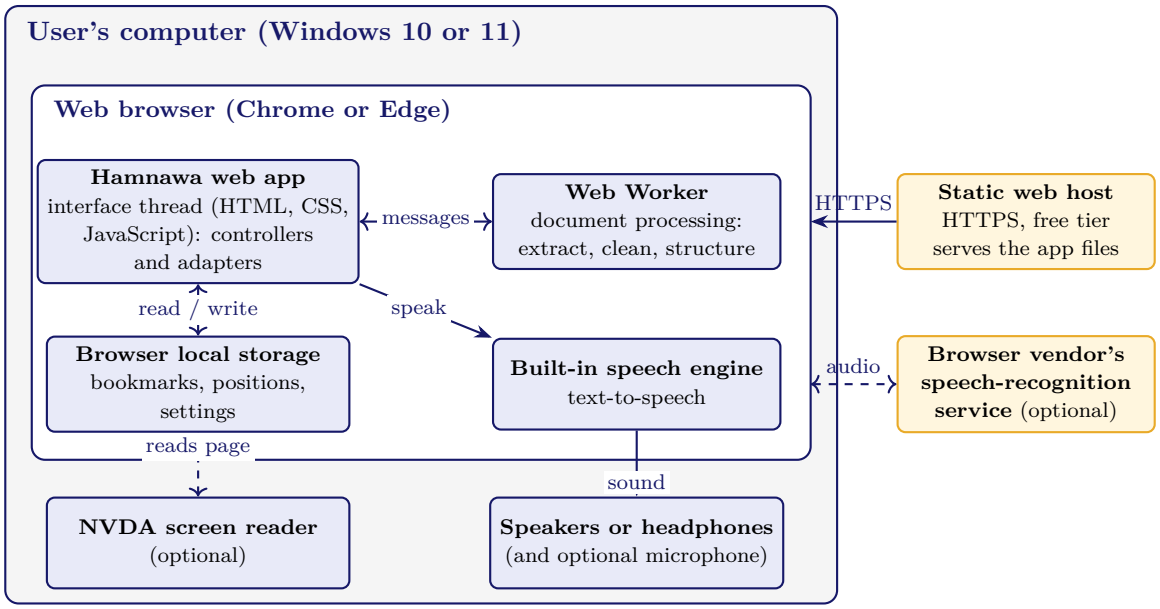


Figure 4: Deployment view. Everything the user touches runs on their own computer; the only network traffic is loading the app and, optionally, speech recognition.

5 Use-Case Model

5.1 Actors

- **Student** (primary actor): a blind or low-vision student who studies from course documents.
- **Browser speech services** (supporting system actor): the speech synthesis and recognition provided by the browser and operating system.

Teachers are not actors: they never use the system, they only supply ordinary documents.

5.2 System-Level Use-Case Diagram

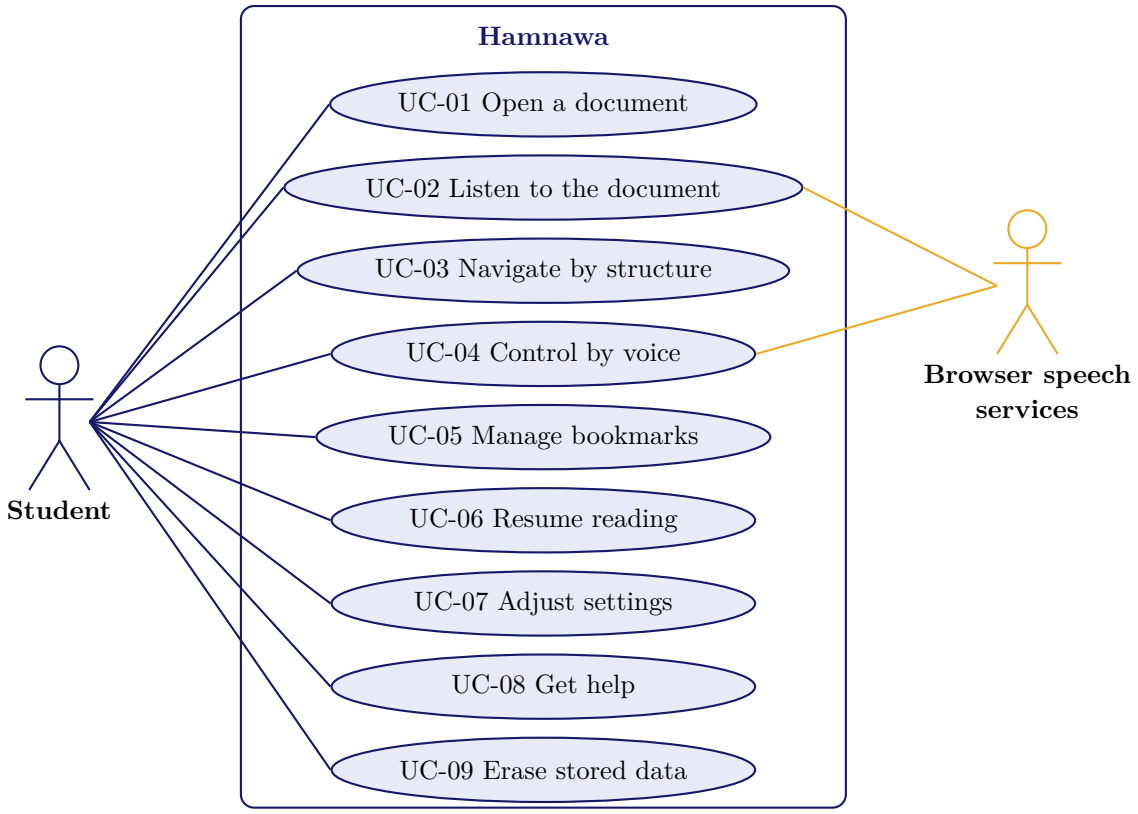


Figure 5: System-level use-case diagram. UC-09 is new and derives from FR-07 (erase all stored data).

5.3 Use-Case Diagram with Include and Extend Relationships

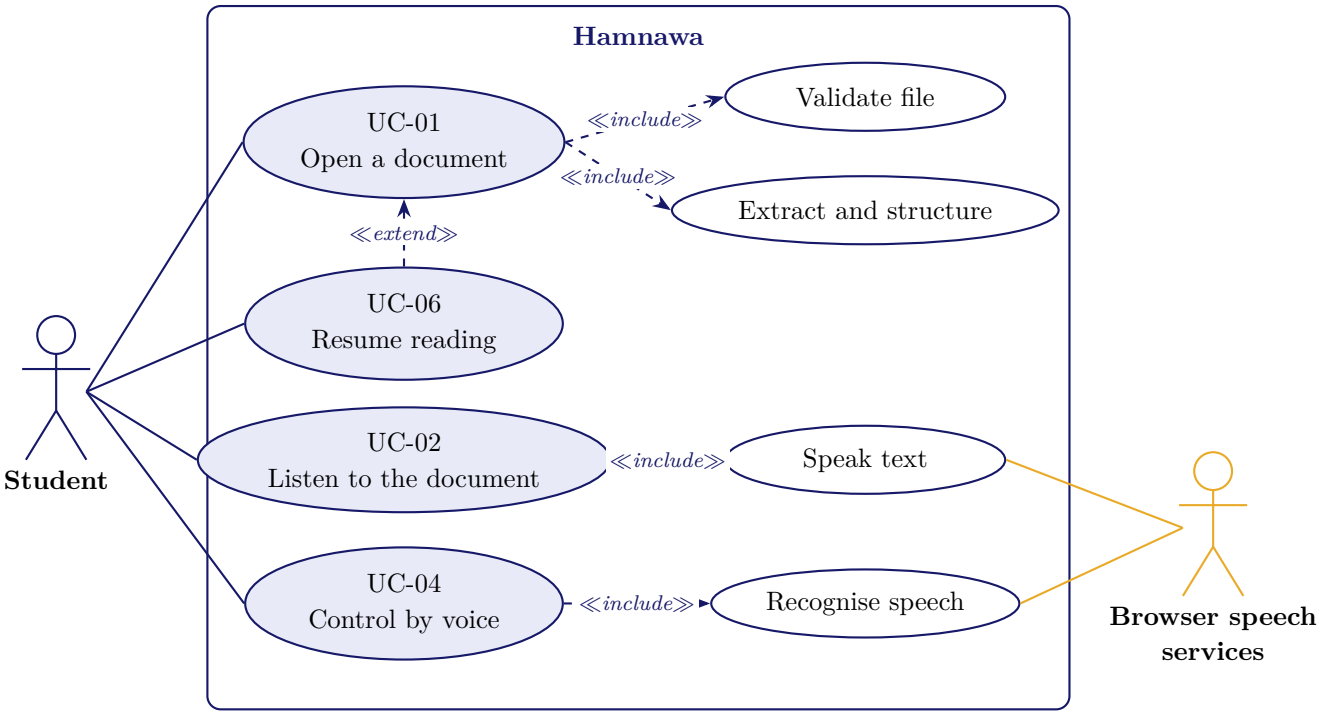


Figure 6: Include and extend relationships for the core use cases. An *include* is behaviour the base use case always needs; an *extend* is optional behaviour added under a condition (resume is offered only if a saved position exists).

5.4 Use-Case Catalogue

Full flows (main, alternative and exception) for UC-01 to UC-05 are in the Software Requirements Report. This table connects every use case to its requirements and delivery increment.

ID	Name	Goal	Requirements	Inc.
UC-01	Open a document	Load a PDF or PPTX and learn what it contains.	FR-01 to FR-05, FR-08 to FR-17	1–2
UC-02	Listen to the document	Hear the content with full playback control.	FR-18 to FR-27	1
UC-03	Navigate by structure	Jump between sections, slides and pages.	FR-28 to FR-34	2
UC-04	Control by voice	Operate the reader with spoken commands.	FR-35 to FR-39	3
UC-05	Manage bookmarks	Mark, list and revisit important places.	FR-40, FR-41	3
UC-06	Resume reading	Continue from the last saved position.	FR-42, FR-43	3

ID	Name	Goal	Requirements	Inc.
UC-07	Adjust settings	Change speed, voice, contrast and text size.	FR-22, FR-23, FR-44, FR-45	3
UC-08	Get help	Hear available shortcuts and commands.	FR-46	1
UC-09	Erase stored data	Remove all bookmarks, positions, history and settings.	FR-07	3

6 Class-Responsibility-Collaborator (CRC) Model

6.1 How to Read the Cards

A CRC card describes one class. **Responsibilities** are what the class knows or does; **collaborators** are the other classes it must ask for help to do so. Each card also lists the requirements it helps to satisfy. We found the classes by taking the nouns from the use cases and requirements, then grouping responsibilities so that every class has one clear purpose. Where two classes depend on each other only through events, the collaborator is the one that *receives* the call.

6.2 Cards

UIController — Presentation layer

Responsibilities	Collaborators
• Render the Open, Reader and Outline regions with correct landmarks and headings • Manage keyboard focus and a logical focus order • Capture keyboard shortcuts and pass them to the right controller • Apply high-contrast mode and text size; highlight the sentence being read • Show and read the help (shortcuts and voice commands)	• PlaybackController • NavigationController • VoiceCommandController • BookmarkManager • SettingsManager • Announcer

Satisfies: FR-01, FR-26, FR-31, FR-44, FR-46, NFR-A1 to A6, NFR-U1, U2

Announcer — Presentation layer

Responsibilities	Collaborators
• Speak and display each status or error message • Queue and prioritise messages (errors first) • Expose text in a live region for the user's screen reader (coexistence mode) • Report progress while a document is processed	• SpeechSynthesizerAdapter • UIController • SettingsManager

Satisfies: FR-04, FR-27, FR-47, NFR-R3, NFR-A4

PlaybackController — Application layer

Responsibilities	Collaborators
• Start, pause, resume and stop reading from the current position • Repeat the current sentence or paragraph; skip sentences and paragraphs • Apply speech rate, voice and volume from the settings • Announce structural cues (heading, list, slide) before the element • Continue into the next section or stop, as set; announce the end of the document • Publish position changes as events	• DocumentModel • SpeechSynthesizerAdapter • SettingsManager • ResumeService • Announcer

Satisfies: FR-18 to FR-25, NFR-P2

NavigationController — Application layer

Responsibilities	Collaborators
• Move to the next or previous section or heading of a chosen level • Go to a slide or page number; jump to the start or end • Open the outline and jump to the chosen entry • Announce the current position on request • Search for a word or phrase and jump to the next match	• DocumentModel • PlaybackController • Announcer

Satisfies: FR-28 to FR-34, NFR-U2

VoiceCommandController — Application layer

Responsibilities	Collaborators
• Keep the microphone off by default; switch it on and off on request • Pause or lower speech while listening • Show the privacy notice before first use • Ask the parser to interpret each phrase and run the matching action • Give a confirmation sound or an error signal • Say so and continue by keyboard if recognition is unavailable	• SpeechRecognizerAdapter • CommandParser • PlaybackController • NavigationController • BookmarkManager • Announcer

Satisfies: FR-35 to FR-39, NFR-P3, NFR-S3

BookmarkManager — Application layer

Responsibilities	Collaborators
• Add a bookmark at the current position with an optional label • List the bookmarks of the open document • Delete a bookmark; jump to a chosen bookmark • Tell the user if a bookmark could not be saved	• DocumentModel • StorageService • NavigationController • Announcer

Satisfies: FR-40, FR-41, NFR-R2

ResumeService — Application layer

Responsibilities	Collaborators
• Save the reading position periodically and on pause or exit • Look up a saved position by document fingerprint • Offer to resume when the same document is opened again • Keep the list of recently opened documents and let the user remove entries	• StorageService • PlaybackController • Announcer

Satisfies: FR-06, FR-42, FR-43, NFR-R2

SettingsManager — Application layer

Responsibilities	Collaborators
• Load saved settings or defaults at start • Save changes (speech rate, voice, cues, contrast, text size, continuous reading) • Notify other components when a setting changes • Erase all stored data after confirmation	• StorageService • UIController • PlaybackController • Announcer

Satisfies: FR-07, FR-22, FR-23, FR-44, FR-45

DocumentLoader — Domain and processing layer

Responsibilities	Collaborators
• Accept the chosen file; check type, size and length • Compute the content fingerprint that identifies the document • Start extraction and structuring in a background worker • Detect unreadable files (scanned, protected, corrupted) and report the reason • Announce title, page or slide count and number of sections when ready	• DocumentExtractor • StructureAnalyzer • DocumentModel • ResumeService • Announcer

Satisfies: FR-01 to FR-05, FR-43, NFR-P1, NFR-P4, NFR-R1, NFR-S4

DocumentExtractor — Domain and processing layer

Responsibilities	Collaborators
• Read text pieces with page or slide, position, font size and weight • Read slide titles, body text and (if chosen) speaker notes • Order multi-column pages column by column • Detect images, tables and charts and note their location • Signal when a file has no readable text	• DocumentLoader • StructureAnalyzer • PDF and PPTX reading libraries

Satisfies: FR-08, FR-09, FR-10, FR-16

StructureAnalyzer — Domain and processing layer

Responsibilities	Collaborators
• Remove repeated headers, footers and page numbers • Repair hyphenated line breaks, ligatures and stray line breaks • Detect headings and their levels from font ranking and numbering patterns • Detect paragraphs, lists (with item count) and slide boundaries • Name untitled slides “Slide N” • Build the outline tree and hand it to the document model	• DocumentExtractor • DocumentModel

Satisfies: FR-11 to FR-15, FR-17

DocumentModel — Domain and processing layer

Responsibilities	Collaborators
• Store the document, its sections and content blocks in order • Return the next or previous block, sentence, paragraph or section • Find a slide, page, heading level or search match • Convert between a position and a location (section order, block index) • Provide summary information (title, counts)	• Section • ContentBlock • PlaybackController • NavigationController • BookmarkManager

Satisfies: FR-15, FR-28 to FR-34

CommandParser — Domain and processing layer

Responsibilities	Collaborators
• Normalise the phrase (case, filler words) • Match it against the command table (play, pause, next section, go to slide n, …) • Extract parameters such as the slide or page number • Return a command, or “unknown”	• VoiceCommandController

Satisfies: FR-35

SpeechSynthesizerAdapter — Infrastructure layer

Responsibilities	Collaborators
• Speak one sentence at a time; report start and end of each • List available voices; set rate, voice and volume • Pause, resume and cancel speech • Detect missing or failing speech and report it	• PlaybackController • Announcer • Browser speech engine

Satisfies: FR-18, FR-22, FR-23, NFR-R3, NFR-C1 to C3

SpeechRecognizerAdapter — Infrastructure layer

Responsibilities	Collaborators
• Ask for microphone permission • Start and stop listening; deliver recognised text • Report when recognition is unsupported or denied • Restart listening after a result or a timeout	• VoiceCommandController • Browser speech recogniser

Satisfies: FR-35 to FR-39, NFR-C3

StorageService — Infrastructure layer

Responsibilities	Collaborators
• Save, read, list and delete records (bookmarks, positions, settings, history) • Erase everything on request • Report storage that is full, blocked or unavailable instead of failing silently	• BookmarkManager • ResumeService • SettingsManager

Satisfies: FR-06, FR-07, FR-40 to FR-45, NFR-R2, NFR-S2

6.3 Collaboration Check

The cards were checked against the use cases by walking through each scenario and confirming that every step is owned by exactly one class. For example, in UC-02 the `UIController` receives the key press, the `PlaybackController` decides what to read next by asking the `DocumentModel`, the `SpeechSynthesizerAdapter` speaks it, and the `ResumeService` records the position. No step is without an owner.

7 Data Model: Entity-Relationship Diagram

7.1 Data Overview

Hamnawa has no server database. Data lives in two places:

- **Stored in the browser** (gold headers in the diagram): document identity, bookmarks, reading position and settings. These survive closing the browser.
- **Built in memory** (blue headers): the section tree and content blocks. They are rebuilt every time a document is opened, so they are never stored.

A document is identified by a *fingerprint* of its content, not by its file name (BR-02). A saved position is a pair (section order, block index), which is stable because the same document always produces the same structure.

7.2 ERD

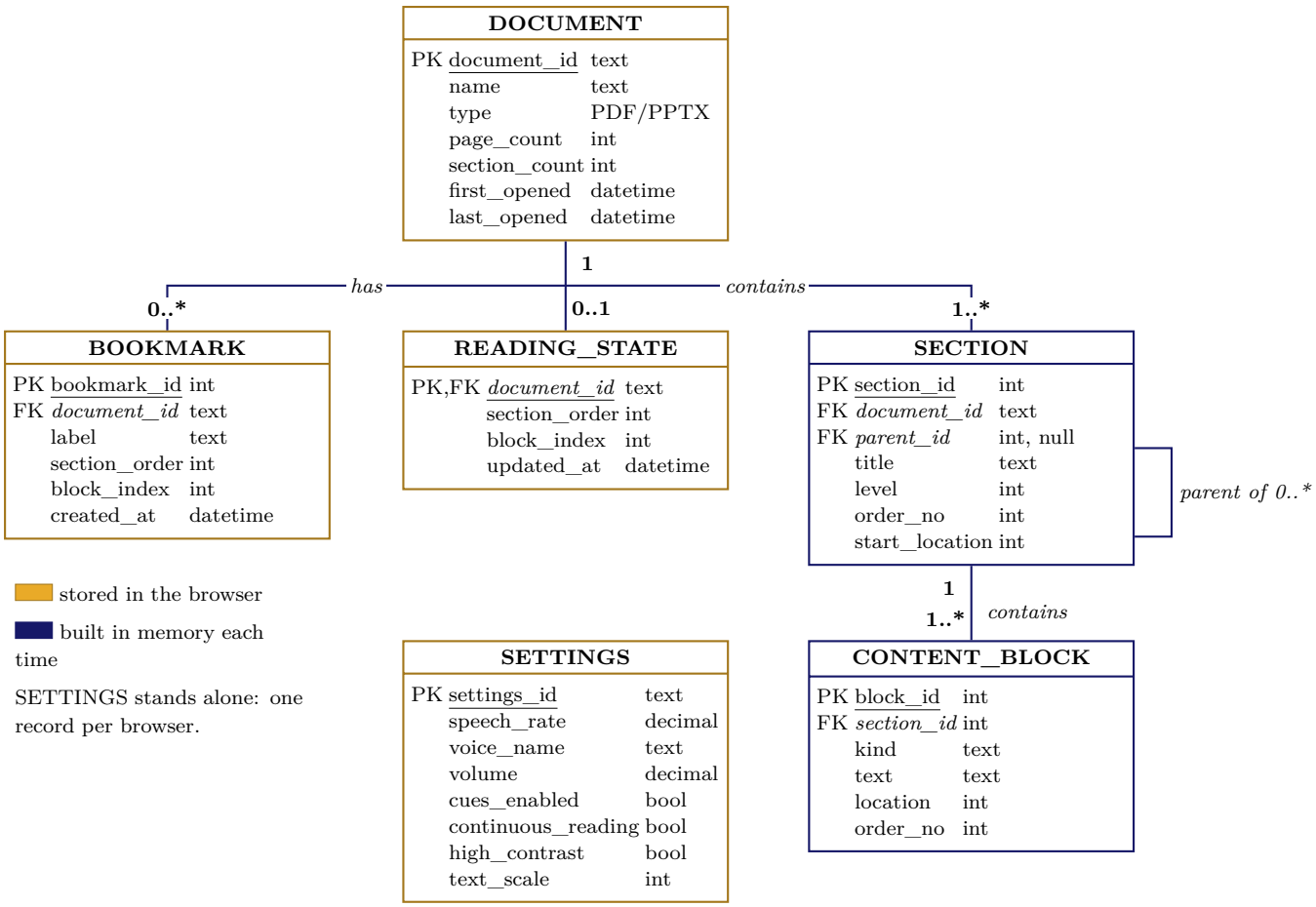


Figure 7: Entity-relationship diagram of Hamnawa. PK = primary key, FK = foreign key (italic). Cardinalities are shown next to the entity they apply to.

7.3 Data Dictionary

Entity	Attribute	Type	Key	Description
`DOCUMENT`	document_id	text	PK	SHA-256 fingerprint of the file content (BR-02)
	name	text		File name as last seen; not used for identity
	type	text		"PDF" or "PPTX"
	page_count	int		Pages or slides
	section_count	int		Top-level sections found
	first_opened	datetime		First time the document was opened
	last_opened	datetime		Used for the recent-documents list (FR-06)
`BOOKMARK`	bookmark_id	int	PK	Generated identifier
	document_id	text	FK	Document the bookmark belongs to
	label	text		Optional typed or spoken label
	section_order	int		Position part 1: order of the section
	block_index	int		Position part 2: block inside the section
	created_at	datetime		When the bookmark was added
`READING_STATE`	document_id	text	PK, FK	One saved position per document
	section_order	int		Position part 1
	block_index	int		Position part 2
	updated_at	datetime		Last time the position was saved
`SETTINGS`	settings_id	text	PK	Always "default" (one record per browser)
	speech_rate	decimal		0.5 to 3.0 (BR-03)
	voice_name	text		Chosen system voice
	volume	decimal		0.0 to 1.0

Entity	Attribute	Type	Key	Description
	cues_enabled	bool		Announce headings, lists and slide numbers
	continuous_reading	bool		Continue into the next section automatically
	high_contrast	bool		High-contrast display mode
	text_scale	int		Text size in percent, 100 to 200
`SECTION (in memory)`	section_id	int	PK	Generated when the document is opened
	document_id	text	FK	Owning document
	parent_id	int	FK	Parent section; null for top-level sections
	title	text		Heading text, or “Slide N”
	level	int		1 is the highest level
	order_no	int		Order among siblings; used as section_order
	start_location	int		Page or slide where the section starts
`CONTENT_BLOCK (in memory)`	block_id	int	PK	Generated when the document is opened
	section_id	int	FK	Owning section
	kind	text		paragraph, list_item, slide_text or notice (image, table, chart)
	text	text		Text to be spoken
	location	int		Page or slide number
	order_no	int		Order inside the section; used as block_index

8 Quality Attributes and Architectural Tactics

Quality	Tactic in the architecture	Where it lives
Usability and accessibility	Semantic HTML with ARIA landmarks; one logical focus order; single-key shortcuts in the reader; every message also announced in a live region; coexistence mode for screen readers.	UIController, Announcer (NFR-A1 to A6, U1 to U4)
Performance	Extraction and structuring in a Web Worker; sentence-level speech queue; outline built once and reused; lazy loading of page content.	DocumentLoader, DocumentExtractor, PlaybackController (NFR-P1 to P4)
Reliability	Defensive parsing with a catch at the worker boundary; every failure routed to the Announcer; storage errors reported, not thrown.	DocumentLoader, StorageService, Announcer (NFR-R1 to R3, S4)
Security and privacy	No document leaves the device; data stored locally only; microphone off by default; notice before first use of recognition.	DocumentLoader, StorageService, VoiceCommandController (NFR-S1 to S3)
Compatibility	Feature detection in the adapters; clear spoken message when a feature is missing; keyboard control always available.	SpeechSynthesizerAdapter, SpeechRecognizerAdapter (NFR-C1 to C3, FR-39)
Maintainability and testability	Layered modules with documented interfaces; adapters replaceable by fakes in tests; format-independent document model.	All layers (NFR-M1 to M3)

9 Traceability

9.1 Components to Requirements

Component	Requirements satisfied
`UIController`	FR-01, FR-26, FR-31, FR-44, FR-46, NFR-A1 to A6, NFR-U1, U2
`Announcer`	FR-04, FR-27, FR-47, NFR-R3, NFR-A4
`PlaybackController`	FR-18 to FR-25, NFR-P2

Component	Requirements satisfied
`NavigationController`	FR-28 to FR-34, NFR-U2
`VoiceCommandController`	FR-35 to FR-39, NFR-P3, NFR-S3
`BookmarkManager`	FR-40, FR-41, NFR-R2
`ResumeService`	FR-06, FR-42, FR-43, NFR-R2
`SettingsManager`	FR-07, FR-22, FR-23, FR-44, FR-45
`DocumentLoader`	FR-01 to FR-05, FR-43, NFR-P1, NFR-P4, NFR-R1, NFR-S4
`DocumentExtractor`	FR-08, FR-09, FR-10, FR-16
`StructureAnalyzer`	FR-11 to FR-15, FR-17
`DocumentModel`	FR-15, FR-28 to FR-34
`CommandParser`	FR-35
`SpeechSynthesizerAdapter`	FR-18, FR-22, FR-23, NFR-R3, NFR-C1 to C3
`SpeechRecognizerAdapter`	FR-35 to FR-39, NFR-C3
`StorageService`	FR-06, FR-07, FR-40 to FR-45, NFR-R2, NFR-S2

9.2 Use Cases to Components

UC	Name	Main components involved
UC-01	Open a document	UIController, DocumentLoader, DocumentExtractor, StructureAnalyzer, DocumentModel, Announcer, ResumeService
UC-02	Listen to the document	UIController, PlaybackController, DocumentModel, SpeechSynthesizerAdapter, ResumeService, Announcer
UC-03	Navigate by structure	UIController, NavigationController, DocumentModel, PlaybackController, Announcer
UC-04	Control by voice	VoiceCommandController, SpeechRecognizerAdapter, CommandParser, PlaybackController, NavigationController, Announcer
UC-05	Manage bookmarks	UIController, BookmarkManager, StorageService, NavigationController
UC-06	Resume reading	ResumeService, StorageService, PlaybackController, Announcer
UC-07	Adjust settings	UIController, SettingsManager, StorageService
UC-08	Get help	UIController, Announcer

UC	Name	Main components involved
UC-09	Erase stored data	SettingsManager, StorageService, Announcer

10 Architectural Risks and Open Issues

ID	Risk or open issue	Mitigation	Owner / when
AR-1	PDF reading order from the library may be poor on complex layouts.	Test corpus; fall back to simple order and warn the user.	[owner], Inc. 1
AR-2	Browser speech engines behave differently and may stop on long text.	Sentence queue; adapter handles errors.	[owner], Inc. 1
AR-3	Speech recognition accuracy and availability vary.	Keyboard is always sufficient; voice is optional.	[owner], Inc. 3
AR-4	Browser-only processing may be too slow for 100-page files.	Worker, progressive reading; revisit decision D1.	[owner], Inc. 1
AR-5	Chosen libraries may have licence or maintenance issues.	Licence review; wrap libraries behind our interfaces.	[owner], Inc. 1
AR-6	Shortcut keys may clash with NVDA or the browser.	Conflict test; allow customising later.	[owner], Inc. 2

A Glossary

CRC card A card describing a class by its Class name, Responsibilities and Collaborators.

ERD Entity-relationship diagram: entities, their attributes and the relationships between them.

Use case A description of how an actor uses the system to reach a goal.

Adapter A class that hides a browser or library feature behind a simple interface of our own.

Web Worker A browser feature that runs code in a background thread.

Fingerprint A short value computed from a file's content that identifies it.

Layered architecture	A style where each layer uses only the layer beneath it.
Pipeline	A style where data passes through a chain of single-purpose steps.
Live region	A part of a web page whose changes are announced by screen readers automatically.